The Cloak of Invisibility

Towards a "Spray Computers" vision

Franco Zambonelli & Marco Mamei

February 2003

(revised March 2006)

Outline

- Part 1: The Vision of "Spray Computers"
  - Spray computers and self-organization
  - Applications of Spray computers
- Part 2: The Invisible Wall
  - Self-localization and routing
  - Technology
  - Applications
- Part 3: The Invisible Object
  - Self-localization and routing
  - Technology
  - Applications
- Part 4: The Cloak of Invisibility
  - The problem of network dynamics
  - Technology
  - Applications
- Conclusions

Can You See?

- A Future in which cans of "smart paints"
  - Will be sold in stores for a few dollars
  - With "computer-based" molecules
  - Capable of self-organizing their activity
  - To provide, when painted, a range of futuristic functionalities
- For example
  - A "spray of invisibility"
  - That when painted on any object or on a cloak
  - Will make the painted object invisible

The Vision of "Spray Computers"

- But it's not only about spray cans and a far future..
- We have already deeply analyzed that in modern distributed computing scenarios we must forget about traditional "deployment"
  - A component (and the associated software) cannot be "placed" in well-defined position in a network
  - The structure of the network cannot be engineered in any way
  - The evolution of the network cannot be exactly predicted
- That is, computers are deployed as in a "spraying" process
  - And "spraying" can act as a powerful metaphor

Generality of the "Spray Computers" Vision

- Literal: Micro-computers:
  - If we have << $1mm^3$ computers, we could indeed produce a paint or a spray
  - to physically distribute myriads of such components in an environment
  - Sensor networks and smart dusts goes in that direction…
- Metaphorical: A "cloud" of persons with a PDA
  - Moving around and interacting via the PDAs → as particles moved by the wind
- Metaphorical: The Internet
  - Node mobility
  - Virtual mobility of Services and ephemeral services (e.g., Gnutella)

Why Self-Organization in Spray Computers?

- We cannot think at "manually" configure the molecules of the spray, rather
- Components should get "sprayed" and then:
  - They recognize who and where they are (w.r.t. the other components)
  - They identify their specific task in the network (according to the who and where)
  - They start working in cooperation with the other components to achieve their task
  - The global goal/configuration is reached without any supervision
- Upon dynamic changes
  - They recognize such changes
  - And re-adapt the initial configuration to suit the new situation

Applying Spray Computers…

- Given the availability of:
  - Theoretical models
  - Algorithms
  - Middleware infrastructures
  - Programming abstractions
  - Methodologies and tools
- For "spray computers"
  - i.e. strongly relying on self-organization and emergent organization
- A number of potential innovative applications can be conceived and deployed….

Application Visions….

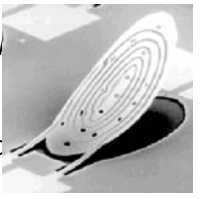
- The "spray" self-organizing Web (that's already on the way…)
  - P2P access to data and services
  - Dynamic and self-healing re-structuring of links
  - Tolerating, e.g., mobility and faults
  - Relying on self-localization on virtual overlay network structures (e.g., Gnutella links)
- And more:
  - Emergence of communication languages and conventions
  - Emergence of peculiar highly optimized structures (e.g., small worlds and scale free)

Application Visions….

- Spray television
  - Micro computer-based emitters
  - To display TV programs and PC screens
  - Requires:
    - Self localization of components, and possibly distributed time synchronization
- Smart paintings
  - To display colors and various patterns on demand
  - Requires
    - Self localization of components, coordinated emergent behaviors

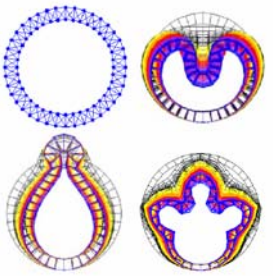
Application Visions….

- Active cereal boxes (remember "Minority Report"?)
  - Painted with optical micro-computers
  - Activated upon movement
  - Start animating a cartoon
  - Requires:
    - Self localization, time synchronization, emergent coordinated behaviors, self-differentiation
- Active pipelines
  - Micro components affecting fluid flow
  - And avoiding e.g., turbulence
  - Requires
    - Coordinated distributed sensing and coordinated movements



Application Visions….

- Self-assembly
  - Micro and nano-scale components
  - Capable of orchestrating their movement
  - So as to assume specific shapes
  - Adaptive and self-healing
- Amorphous computing
- Terminator 2



(from Comm. ACM, May 2000)

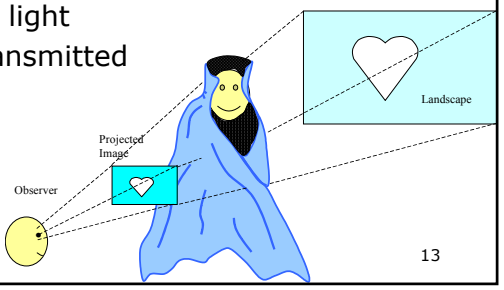


The Chricton's Vision in "Prey"

- "Swarms" of nano component
  - Mixed organic and silicon-based material
  - Capable of flying by attaching to air molecules
  - Capable of self-assembly
  - Capable of collective self-organizing vision
  - Capable of collective sentient behavior
- Built by a complex self-organized systems of other micro-components
  - The "assemblers"
  - Capable of self-reproduction
- Is this really science fiction?

The Cloak of Invisibility

- A fabric of small computing devices that
  - Gets densely painted (or "sprayed") on a tissue
  - Interact via short-range wireless comms.
  - Can sense and retransmit light emissions in a directional way
- So that:
  - Any blocked ray of light
  - Gets properly retransmitted



Why 'The Cloak of Invisibility'?

- Fascinating application by itself, but….
- The software challenges to build such an artifact are archetypal for a whole range of other application scenarios:
  - E.g., self-localization, routing in sensor and mobile ad-hoc networks, P2P computing, pervasive computing
  - And will enable us to "put at work" a variety of – both direct and reverse engineering – self-organization approaches that we have analyzed during this course
- Here we attempt to highlight these challenges
  - propose a conceptual solution to build the cloak (and other smart artifacts)
  - present a few key algorithms for "spray computers"
  - compare with related approaches

The Clock of Invisibility: Structure of the Presentation

- Incremental presentation: from simple to complex
- The Invisible Wall
  - Provide invisibility for a regular shaped rigid object, from a single fixed point of observation
  - Key issues: 2-D localization, location-based routing
- The Invisible Object
  - Provide invisibility for a rigid object of whatever shape and from whatever point of observation
  - Key issues: 3-D self-localization, content-based routing
- The Cloak of Invisibility
  - Provide invisibility for a flexible fabric from whatever point of observation
  - Key issues: Dynamic re-localization, Routing on mobile ad-hoc networks

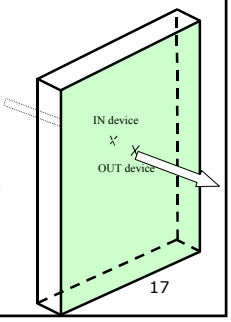
Part 2:

- The invisible Wall

The Invisible Wall

- Paint the wall with densely packed micro devices
  - IN (light sensors) and OUT (light emitters) devices on the two sides, respectively
  - Computer-based
  - Communication-enabled (e.g., IR or radio-based)

- In the sensor network
  - Route messages (light information) from IN to OUT devices on the corresponding coordinates of the wall
  - So as to globally reproduce the image



Software Issues

- Let us assume the sensor and emitters are "painted" (or "sprayed" on the wall)
- To properly establish the IN and OUT pairs
  - devices must **determine their location** on the wall
  - Properties required:
    - **Decentralized and fully autonomous** process
    - **No direct human intervention**
- To route data from IN to OUT devices
  - Specific **routing algorithms** are required
  - Properties required:
    - **Fault-tolerance** (sensors can die or get out of power)
    - **Adaptivity** (we are not looking for a specific sensor, but for the best suited one, I.e., the one closest to the specific coordinates)

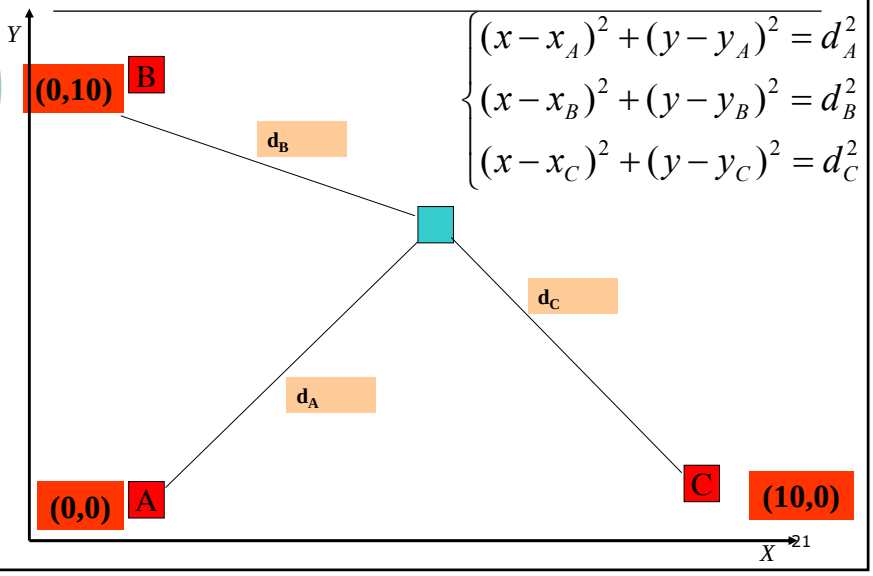
Self-Localization

- **(What is)** Devices in an amorphous sensor network determines their coordinates accordingly to a common reference frame
  - Two steps:
    - Determine the reference frame
    - Determine the location in the frame
- **(What is for)** Spatial coordination, mark data with spatial information, location-aware applications
  - In the cloak, **location-based routing**
  - Send this *color* to device at ***(x,y)*** **coordinates** on the other side of the wall

Localization Principles

- Simple triangulation based mechanism
  - Rely on "beacons" to define a coordinate frame
    - 3 not-aligned beacons needed for a 2D frame,
    - 4 not-aligned beacons needed for a 3D frame.
  - Devices evaluate their distances from the beacons
    - Simple Euclidean considerations lead to the position
    - Distance "triangulation"
- Iterative triangulation
  - When not all devices can estimate their distance from the beacons (I.e., in the presence of short-range communications or obstacles)
    - The devices that have already determined their location
    - Can act as delegated beacons for other devices
    - And so on…

Beacon-based Localization: Example



$$\begin{cases} (x-x_A)^2+(y-y_A)^2=d_A^2 \\ (x-x_B)^2+(y-y_B)^2=d_B^2 \\ (x-x_C)^2+(y-y_C)^2=d_C^2 \end{cases}$$

Identifying Beacons

- External beacons
  - GPS (Global Positioning System)
    - Only outdoor
    - Expensive hardware
  - Wi-Fi access points
    - Reduced accuracy (meters)
    - Suitable for locating, e.g., persons in rooms, not for self-localization of small sensors networks
- Elected beacons
  - Cheap hardware
  - **Suitable to spray computers….**dense networks of peers:
    - Beacons are nodes as the other ones
    - Enable iterative localization

Electing Beacons

- It is basically a "leader election" process
- External stimuli
  - Specific beacons are selected from the external of the network
  - Requires human intervention
  - Difficult for "sprayed" computers
- Autonomous ***Leader Election***
  - Devices generates, e.g., a random number
  - These numbers get broadcasted in the network
  - After a while, the devices having generated the highest numbers will know they are the leaders

Leader Election as a Case of Direct Self-Organization

- Here we use "leader election" to dynamically identify which nodes should act as reference frame
- In general, leader election algorithms are widely used in dynamic distributed systems scenarios
  - To break simmetry in a cloud of identical components, as this may needed to
  - identify who should perform a specific action, when the action should be performed only once
  - Identify who should access a specific resource and e.g., update it
- The process is a classical case of "direct self-organization"
  - We know the process will converge into identifying a single component
  - We do not care who this will be (also to improve robustness: the leader by definition will be alive and participating)
  - We let the components decide by themselves
  - We can identify by design distributed algorithms that stabilize with the identification of such leader

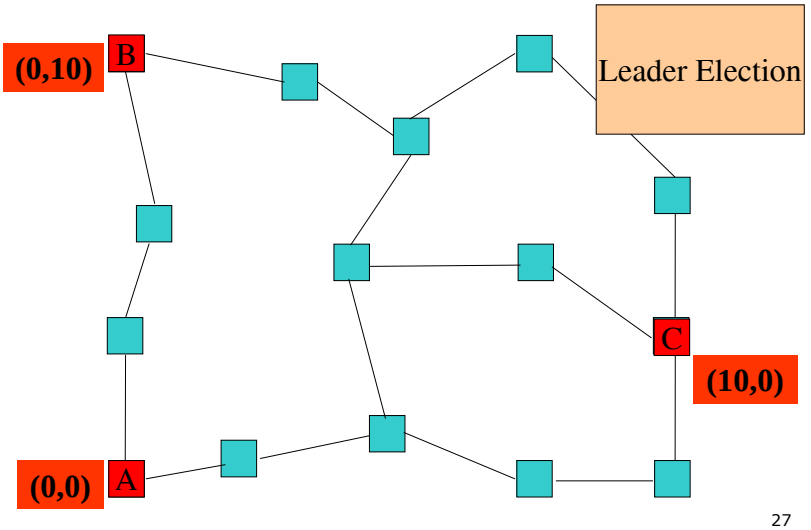
Estimating Distance from Beacons

- Principle
  - Beacons emits a signal to neighbor devices (e.g., those in the communication range)
  - Devices determines distance on the basis of specific properties of this signal
- Mechanisms
  - Signal attenuation (e.g., radio signal)
    - Not enough precision on short distances
  - Travel time of acoustic waves or phase displacement of IR waves
    - Could be more accurate (cm-scale obtained with acoustic waves and IPAQs)
    - Requires very accurate time synchronization (which is another challenging problem by its own)
  - **Density based approaches**

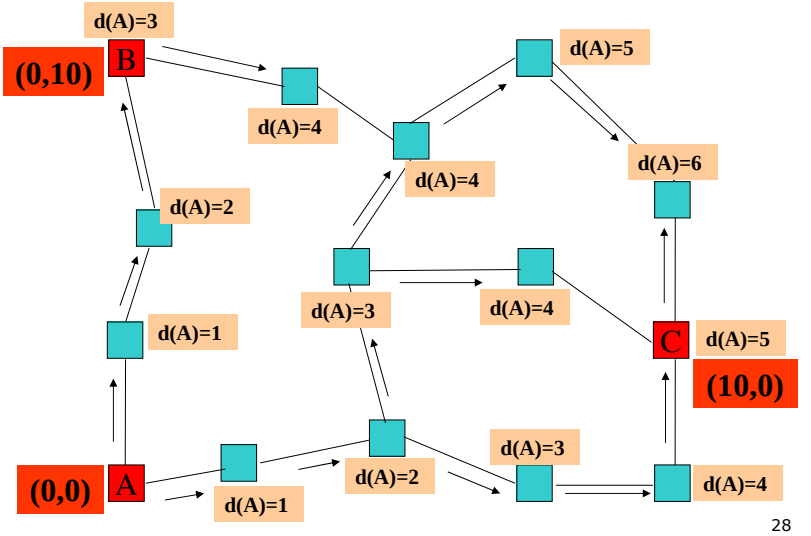
Density-based Distance Estimation

- Requires Devices
  - to be "dense" and to know their average density
  - to be able to send short-range signals to close devices
- Then
  - The beacon send a message to neighbors with a integer = 1, counting the number of hops the message has traveled
  - The signal is re-transmitted by each device after having incremented the counter
  - After a while, devices know their distance, in terms of hops, from the beacons
- Eventually
  - Triangulate and Calculate *Hops*AvgDist = PhysDist*
- Accuracy
  - Depends on average density: *Accuracy=AvgDist*
  - Requires at least 15 one-hop neighbors
  - Could be improved by multiple iterations

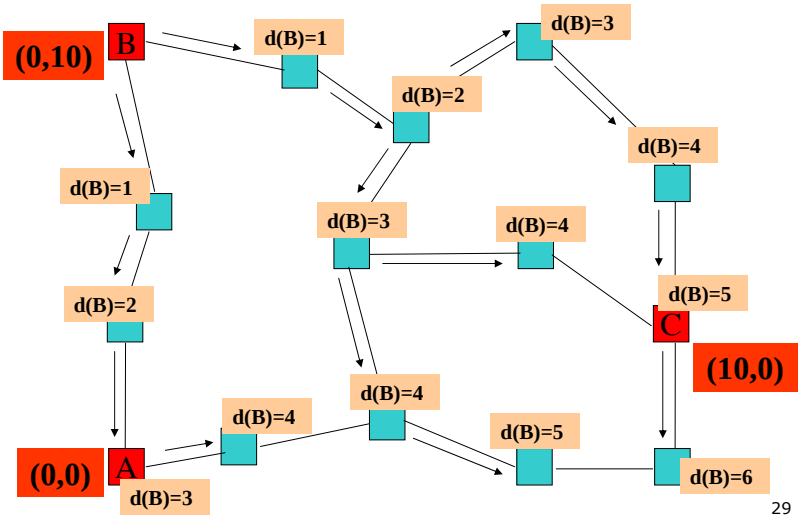
Example



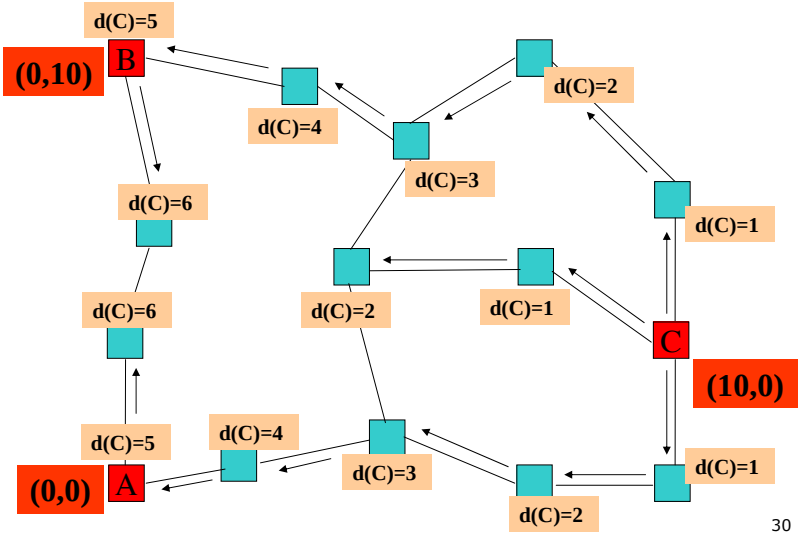
Example



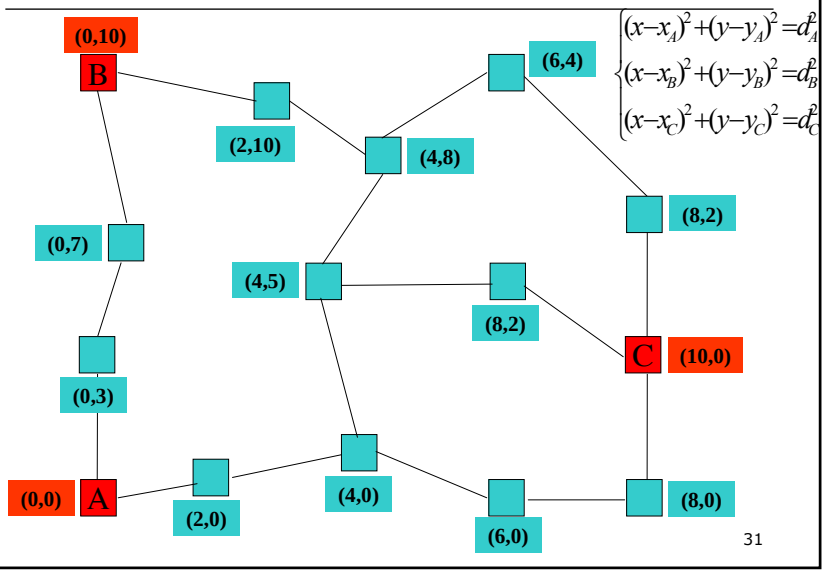
Example



Example



Example



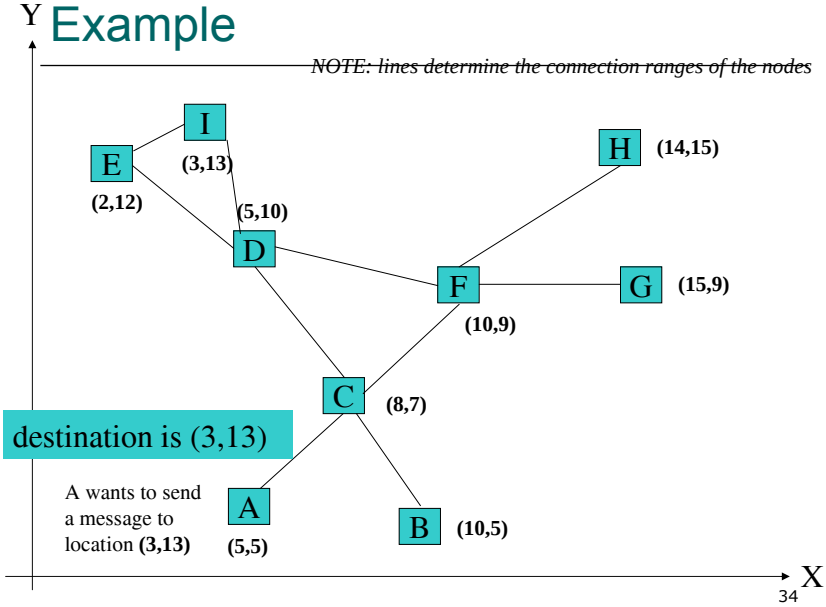
Routing Issues…

- Once devices know their location…
- How can you route data across the network from a device to another device at a specific location?
  - E.g., in the wall of invisibility, from an IN device to the OUT devices on the opposite side of the wall?

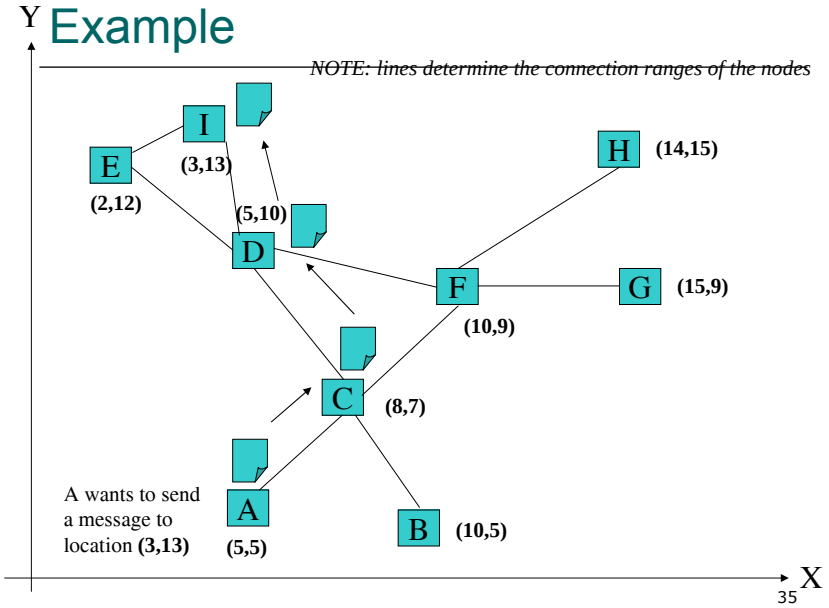
Location-Dependent Routing

- Take advantage of devices' location knowledge to route information
  - Information sent to a particular location, rather than to a particular device
- Each node knows its and one-hop neighbors' coordinates in a frame common to all the network
  - Physical space
    - the case of the wall of invisibility, or of mobile ad-hoc networks and sensor networks
  - Virtual Space:
    - The concept of overlay networks in the Internet
- If the space is a continuum
  - The message is propagated in the network on the basis of simple Euclidean considerations
  - To devices closer and closer to the goal

Example



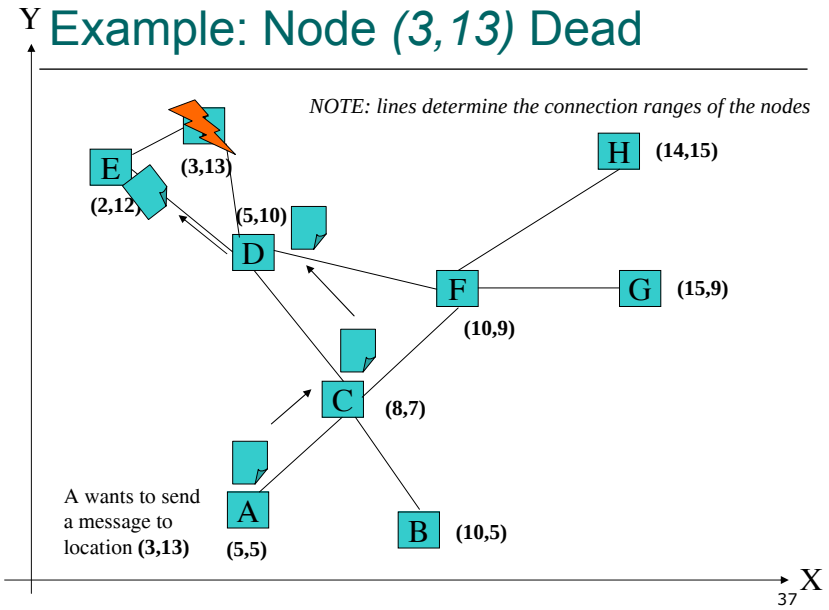
Example



Advantages of Location-based Routing

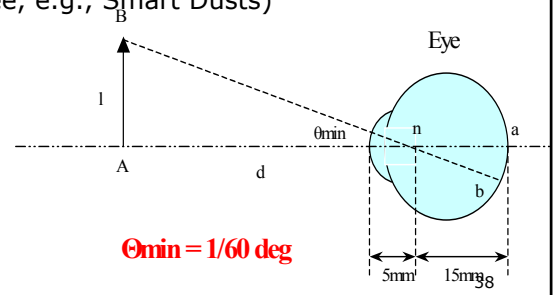
- The message is directed "closer" to the right location
  - Reaching the "closest" locations the goal
  - Suits uncertainties (what really matter is that the message arrive "close" to destination)
  - Suite network dynamics (nodes can move or die and the message will in any case arrive at a proper destination)
- In the wall of invisibility:
  - Do not matter if there is not a sensor at the exact location
- In the Internet
  - Different approaches proposed for virtual overlay networks based on virtual physical spaces (e.g., Pastry defines a virtual 1-D, CAN defined a generic N-D space)
  - If similar services are mapped on close portion of the virtual spaces
    - One do not need to know the IP of a server nor it requires that a specific server is on
    - Just need to reach the zone close to that service….

Example: Node *(3,13)* Dead



Back to the Invisible Wall: Hardware Issues

- How many devices and how small?
  - The Listing-Donders model of the human eye
  - to rend invisible a *1m²* wall from a distance of *10m*, you'd need each device to be approx. *8.4mm²* wide and capable of *286 kbits/sec* wireless bandwidth (30 frames/sec)
  - No problems! (see, e.g., Smart Dusts)



The Invisible Wall: Applications (1)

- Spray monitors and TVs
  - Imagine walking around with a "mini-PC" without monitor, and spraying a monitor in any wall or desk you need it!
    - Just spray the proper can
    - Add a receiver (that will also act as a beacon in the reference frame)
    - Self-establish a coordinate frame
    - And have all data bits be properly routed to the right bits
  - Synchronization may also be important in this context
- Animated posters
  - As in "Minority Report" cereal boxes
  - In this case, the receiver may not be needed
    - The animation to be displayed may be in the memory of components
    - Motion sensor can "wake up" sensor
    - And have them start the animation

The Invisible Wall: Applications (2)

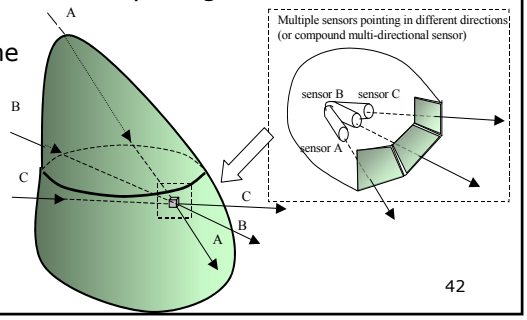
- Smart Paints
  - Imagine painting you room's wall with a smart paing
  - Capable of showing a range of nice images or movies
  - Simply by having them stored in the memory of paint molecules
  - Or better…
- Imagine that the smart paint component act as a cellular automata
  - There is no need of memory
  - Your wall will be able to display a variety of nice "70s style" phsychedelic patterns
  - Simply by executing simple CA rules…
- What about augmented reality?
  - Reproducing reality behind a wall
  - Plus whatever kind of "special effect" to it
- Any other suggestions?

Part 3: The Invisible Object

- The Invisible Object

The Invisible Object

- You no longer separate the object into IN and OUT sides
  - Both IN and OUT devices densely packed in the surface (or packed in a single compound device)
  - Painted with a random orientation, so as to probabilistically have sensor in "all" directions
  - Possibility to reproduce "all" ray of lights incident on the object
    - For any point of the surface
    - For any direction of observation



Software Issues: Self-Localization

- Route light information from one device to the co-aligned device on the other side of the object….
- More than 2-D self-localization on the surface:
  - Each device must know its **coordinates w.r.t. to a 3D reference frame** attached to the object
  - And its **orientation**
  - So as to determine which ray of light it blocks
  - And to determine its "mate", i.e., the device to which to send the light information to reproduce
- And this has to be done:
  - Without any a priori assumption or global knowledge about the object shape
  - Without any a priori global knowledge of devices' orientation

Evaluating Surface Coordinates: 3D Triangulation

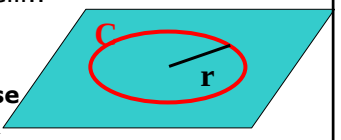
- Potentially possible to extend triangulation mechanism
  - By using 4 non-aligned beacons
  - And by triangulating over a 3D space
- But this cannot be generally applied!
- **Problems**
  - For short range communications, the process must be iterative
  - Devices, in turn, must act as beacons to have the evaluation of coordinates propagate
  - However, in the case of nearly flat portions of the surface
    - One cannot find 4 non-aligned devices to act as beacons!!!
    - This can produce errors!

Evaluating Surface Coordinates: Curvature

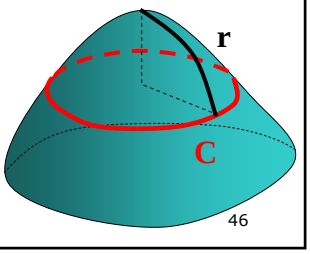
- A completely different approach:
  - 2D triangulation **AND**
  - Curvature estimation
- In particular
  - Devices, other than reciprocal distances, evaluate the local curvature of the surface
  - this enable to reconstruct iteratively the global shape of the surface and accordingly,
  - Determine the 3D position of a device: ***(x,y,z)***
- Problem: How can a device "painted" on a surface determine the surface local curvature?
  - It cannot rely on any external 'view' of the surface to see if the surface locally bends…

Theoretical Solution

- Manifold geometry suggest how to estimate curvature even without an external view.
  - The ratio of the the circumference to the circle's radius changes with the curvature…..
  - In a flat surface: **C/r = 2π**
  - In a curved surface:
    C/r decrease as curvature increase



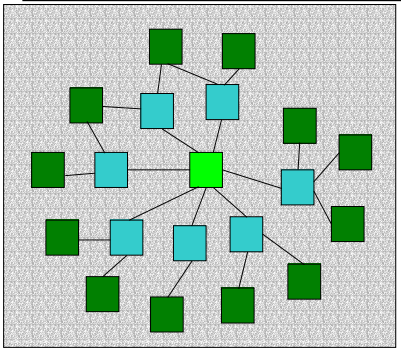
- Thus, each device in the network has to:
  1. Evaluate the circumference of a circle centered in the device (**C**)
  2. Evaluate the radius of the circle (**r**)
  3. Compute **C/r** see from this how far **C/r** is from **2π** to evaluate the curvature



Implementation

- If the network is ***dense*** and with a ***uniform density*** **C** and **r** can be evaluated counting the number of devices on the circle and on the radius…
- Each device send a hop-count broad cast message that spread until an intended fixed number of hop **r**.
- Devices receiving the hop-count message with hop-count = **r** send a message back to the original device.
  - **N.B.** These device are the ones on a circle of radius **r** centered on the original device.
- The device counting the number of replies can evaluate **C**

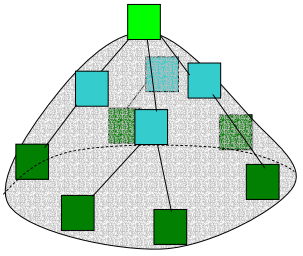
Example



$r = 2$ $C = 12$

$C/r = 6 \sim 2\pi$

Almost Flat



$r = 2$ $C = 6$

$C/r = 6 \sim \pi$

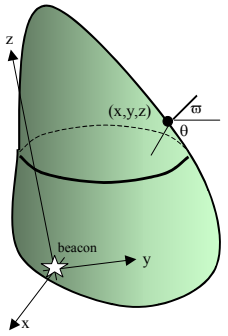
Curved

More on Curvature Estimation

- Several other intrinsic curvature measures…
  - E.g. Area / radius.
  - Performance measures needed on real world systems → still missing.
- Other potential applications
  - Using sprayed sensor networks
  - Monitoring stresses and deformations in artifacts (e.g., skyscrapers, bridges, airplane wings)
  - Monitoring evolution of "turbulences" in fluids
- In general, manifold geometry can provide useful algorithm to dynamic networks and to to smart and self-assembly artifacts

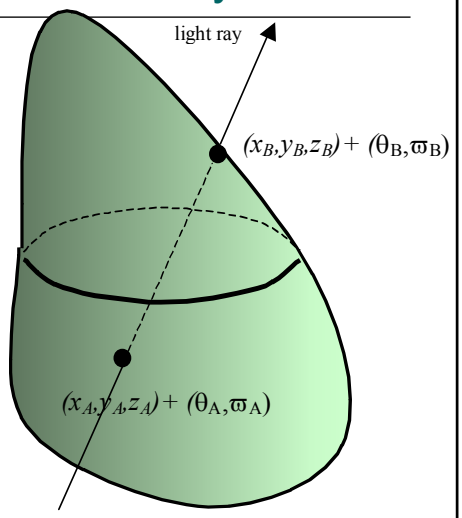
Determining Devices Orientation

- Devices need to know the direction to which they are directed in a common frame.
  - Remember they have been "sprayed" on the surface with a random orientation
- Approaches
  - Beacons also act as orientation reference
  - Estimate relative orientation between two devices
    - Ray-based communication (Infra-Red and phase shift)
    - Gyroscopes?
  - From beacons, propagate and adjust this relative measure by taking into account curvature information



Self-localization: Eventually…

- Each device on the surface **KNOWS**:
  - Its coordinates **(x,y,z)**
  - Its orientation **(θ,ω)**
- And accordingly:
  - The coefficients of the ray of light it block or it has to reproduce

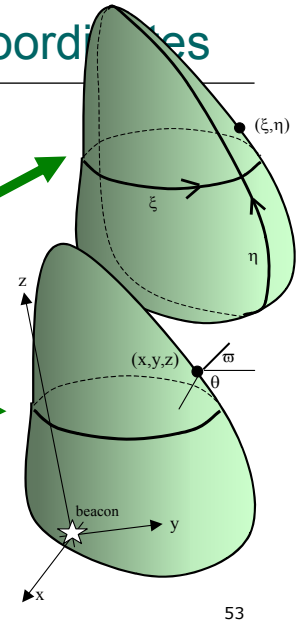


Software Issues: Routing on the Invisible Object

- This is a bit more challenging than in the case of the invisible wall…
- Problems:
  - Even if a device knows
    - WHICH ray of light it blocks
  - It does not know
    - WHERE the "mate" devices that should reproduce it is on the object and in which direction a message should go to reach it
    - This may strongly depends on the surface on the object
    - There is no way to be sure that a direction is correct
  - We cannot certainly think at "flooding" messages over all the network….

Intrinsic Vs. Extrinsic Coordinates

- Terms coming from differential geometry
- Useful to understand the routing problem
- **Intrinsic coordinates**: coordinates ON the surface ***(ξ,η)***
- **Extrinsic coordinates**: coordinates with reference to an external fixed frame ***(x,y,z)*** – better, taking into account orientation too ***(x,y,z)+(θ,ω)***



Routing with Extrinsic and Intrinsic Coordinates

- **Extrinsic coordinates**
  - **PRO:** Code all the information related to the actual shape of the surface (and so of the ray of light associated with a device).
  - **CONS:** It is difficult to route information toward a specific point without global knowledge of the surface → cannot be used for the invisible object
- **Intrinsic coordinates**
  - **PRO:** allows to route information by using simple Euclidean considerations (as the location-based routing in the invisible wall)
  - **CONS:** Gives no information to the actual shape of the surface (and so, gives no information on where to find the needed device)
  - **PS** intrinsic coordinates self-localized easily (2D coordinates!!!)

Routing in the Invisible Object: a Solution

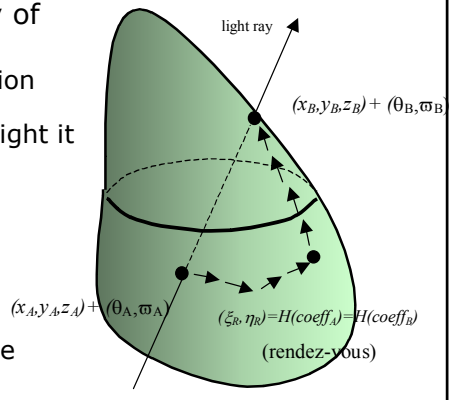
- Exploit the intrinsic coordinates by
  - Using a function ***H*** (*hash*) that maps extrinsic coordinates into intrinsic ones

    $$(\xi,\eta) = H(x,y,z,\theta,\omega)$$

  - Route a message towards the derived intrinsic coordinates (location-based routing, as already described)
- The derived intrinsic coordinates determines a devices that act as a "***rendez-vous***" point
  - A sender send a message on the hashed coordinates of the receiver
  - The receiver go looking for messages on its hashed coordinates

Rendez-Vous Communication

- When a sensor block a ray of light
  - It code the color information and send it to the hashed coordinates of the ray of light it has blocked
- When an emitter has to reproduce a ray of light
  - It goes looking for color information at its hashed coordinates
  - Once obtained, it route the information backwards
- In a convex object the only two devices having that same coefficients are the communicating partners.



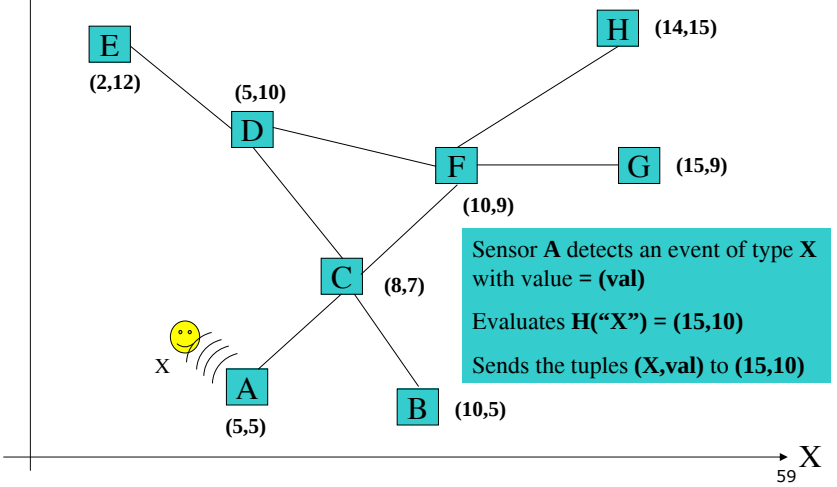
Generality of the Proposed Approach

- The proposed approach for the invisible object is a **specific instance of a more general approach** for content-based routing
  - I know the characteristics of some data I look for (or the "name" of a receiver to which to send data)
  - But I do not know "where" data (or the receiver) is
- Solution
  - Exploit some types of ***intrinsic coordinates*** in which I know how to navigate and how to locate
    - Coordinates of physical space or of some virtual space mapped onto the physical space (or onto an "overlay network)
  - Hash content of data (or name of receiver) into intrinsic coords
  - Use the hashed coordinated as a rendez-vous point
- Applications:
  - Sensor networks & P2P Internet data access
- Inherits the **advantages** of location-based routing
  - Adaptivity and fault tolerance

Content-based Routing on a Physical Space: GHT (UCLA)

- Sensors on a 2D landscape
  - Self-localized and knowing the coordinates of their neighbors
  - In charge of monitoring specific environmental data and exchanging such information with each other
  - All the nodes shares a common hash functions mapping sensed-data-keywords to locations
- Use Location Dependent Routing and hash tables to gather data from an amorphous sensor network
  - A Node sensing data of type X sends the information to location H(X) via location based routing
  - A Node looking for data of type X query node at H(x)…
  - X could be a structured type, or a tuple (associative access)
  - When a query succeeds, the obtained data is routed backwards to the querying node

Example

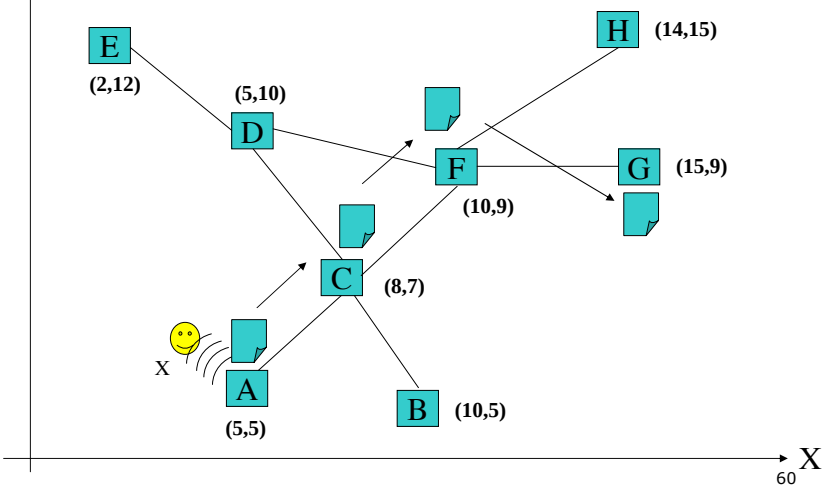


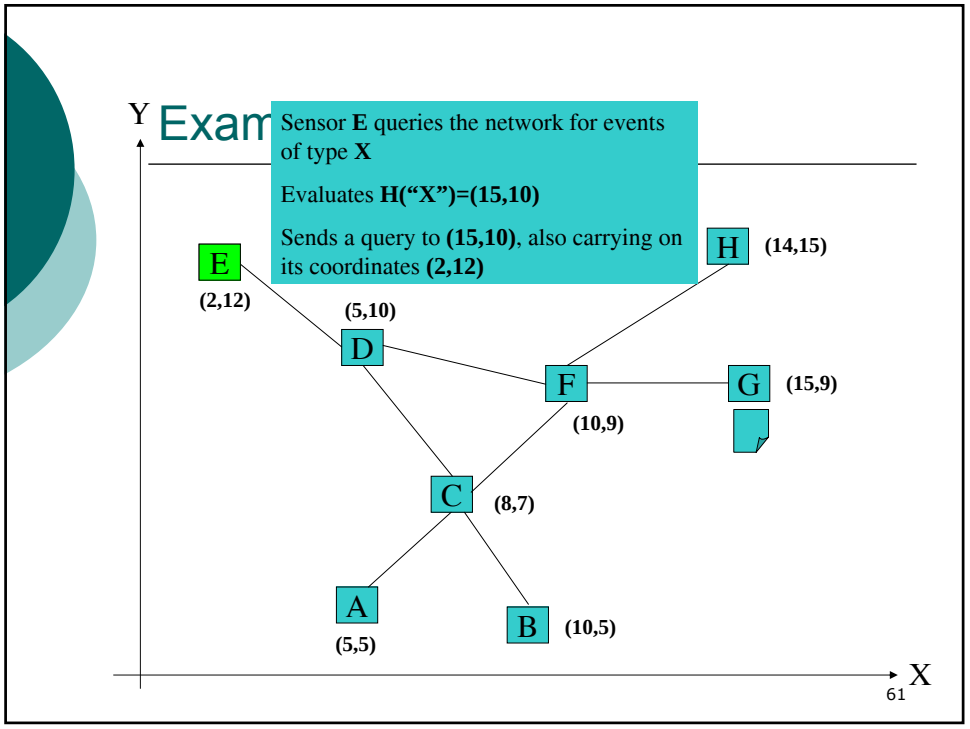
Sensor **A** detects an event of type **X** with value = **(val)**

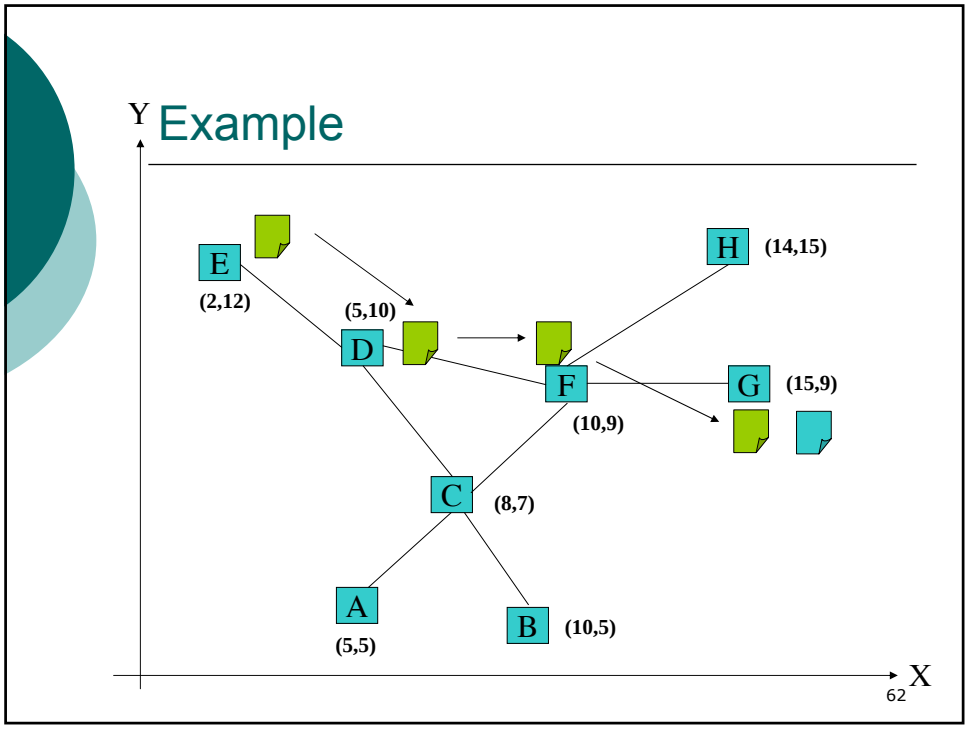
Evaluates **H("X") = (15,10)**

Sends the tuples **(X,val)** to **(15,10)**

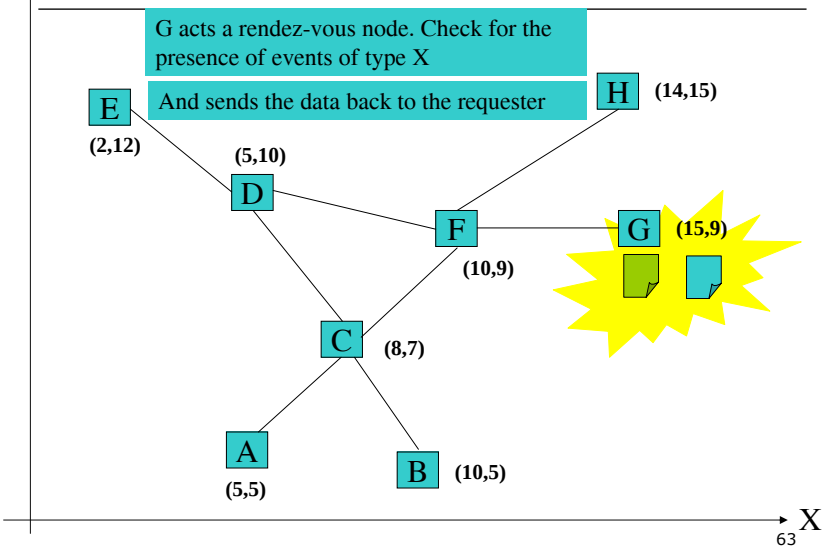
Example



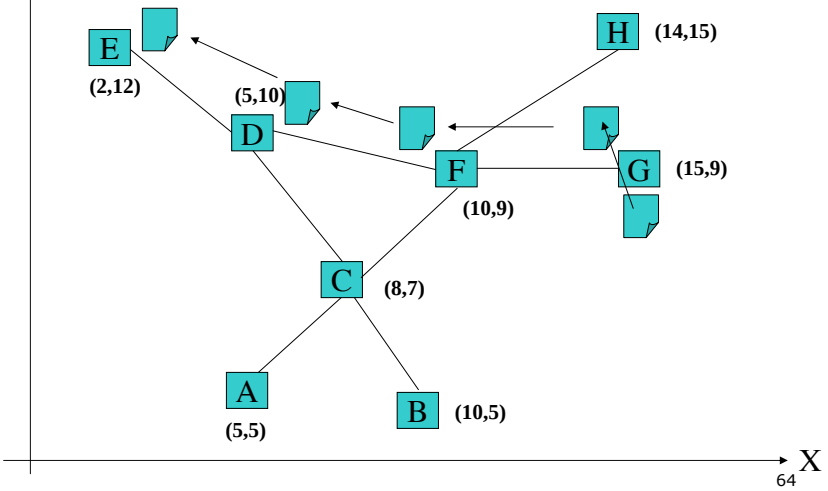

Y Exam
Sensor E queries the network for events of type X
Evaluates H("X")=(15,10)
Sends a query to (15,10), also carrying on its coordinates (2,12)
E
(2,12)
(5,10)
D
H
(14,15)
F
G
(15,9)
(10,9)
C
(8,7)
A
(5,5)
B
(10,5)
X
61


Y Example
E
(2,12)
(5,10)
D
H
(14,15)
F
G
(15,9)
(10,9)
C
(8,7)
A
(5,5)
B
(10,5)
X
62

Example



Example



Content-based Routing on the Internet

- CAN (Berkeley), Pastry (MS Research), Chord (MIT)
- All relying on the same concept of "overlay network"
  - A virtual network structure built over the physical Internet
  - Virtually Connecting nodes according to specific rules
  - The overlay network defines a sort of virtual space, and nodes are connected if adjacent in the space
  - So as that it is possible to navigate in this network
- Example:
  - Pastry & Chord: nodes logically connected in a ring → a 1D logical space
  - CAN: nodes virtually associated to a region of n-D space, each nodes virtually occupying such region and connected to nodes on adjacent regions

Hashing for Content-based Routing on the Internet

- Peer share an hash function ***H***
  - mapping strings (or, in general, some sort of "content") into virtual space coordinates
  - Data (or messages) with a specific content is sent to the the position ***H(content)*** in the virtual spaces
  - Requests for data with a specific content are looked for at the position ***H(content)***
- Example: mp3 file exchange
  - When a peer connect, it is assigned a position in the overlay network and a set of neighbors→ some sort of "space" balancing is enforced
  - Should some nodes die or disappear, the structure is automatically updated
    - nodes recognize a neighbor is dead and re-distributed in the virtual space, to reoccupy the portion of the space left free
  - A peer having the song *"Hey Jude"* sends a tuple *("Hey Jude",IP)* to peer located at *H("Hey Jude")*
  - A peer looking for song *"Hey Jude"* queries device at *H("Hey Jude")* finds *("Hey Jude",IP)* and starts downloading.

Back to the Invisible Object: Hardware Issues

- To rend invisible a *1m*-diameter sphere from a distance of *10m*, you'd need
  - ***372,000* compound objects**. Each
  - composed by ***186,000* mono-directional devices**
  - each ***5μm*5μm*** wide
  - with a ***3.4 Mbits/sec*** communication channel.
- Impossible???? Let's say challenging…
  - Texas Instruments, produced micro-displays made up of electro-statically actuated mirrors of a few m^2
  - Philips Research Laboratories, showing the possibility of growing micro-scale LCD cells on any type of surface
  - Terahertz-band wireless communications potentially possible on silicon

The Invisible Object: Applications

- Well, the potential applications are limited only by fantasy
- Invisible cars and tanks
  - James Bond "Tomorrow Never Die" claims a similar technology!
- Improving visibility
  - In cars and trucks, by avoiding blind spots
  - In mountain slopes, by painting the landscape
- Virtual windows and trompe l'oeil
  - To have windows where this is not possible (e.g., in historical houses)
- Immersive virtual reality environments
  - To paint reality and have it enriched with any kind of virtual rendering
- Any more suggestions?

Part 4

- The Cloak of Invisibility

The Cloak of Invisibility: Software Issues

- Remove rigidity constrain
  - Deploy the network of devices on a flexible fabric, re-shaping due to unpredictable dynamics (e.g. wind and wearer's movements)
- Problems
  - Extrinsic coordinates of devices changes continuously → **dynamic re-localization**
  - Communication partners (IN and OUT pairs) change continuously, and so the route paths
- Similar challenges found in MANET
  - Well, the cloak is a MANET indeed….

Dynamic Re-localization

- Re-compute the self-localization process periodically
  - it would make it possible to leave the location-based routing protocol unchanged
  - Feasible depending on involved dynamics
    - In the cloak, it appears unfeasible (30 frames/sec)
    - Also very expensive
- Base the coordinate system on a fixed point for geometrical references
  - evaluate coordinates as displacements to the fixed ones
  - In the cloak, these could be the belt or a necklace
  - Forces load unbalanced in the cloak

Routing on Mobile ad-Hoc Networks

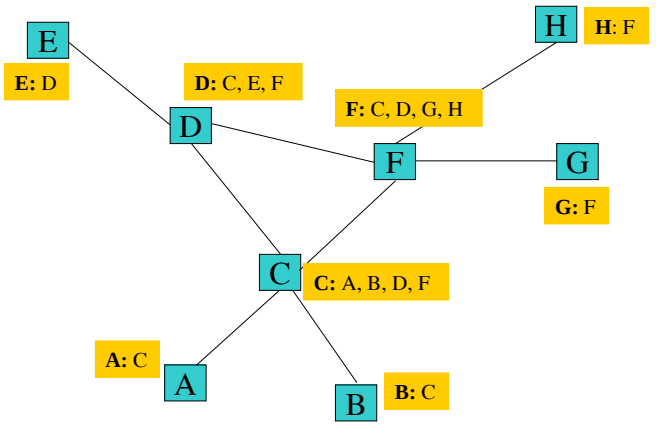
- How are this problems faced on MANETs? (Mobile ad-hoc Networks)
  - Clouds of computer-based nodes (e.g., PDAs)
    - continuously changing their position (e.g., because carried on by moving persons)
    - With short range connections
  - And where messages must be sent from a node of the network to another nodes
    - possibly requiring multi-hopping and intermediate nodes retransmissions
- A number of routing protocols are getting proposed
  - Table-driven, On-demand, adaptive location dependent protocols, gossip algorithms…
- Let's see how they work and how they could potentially apply to "spray computers" and to the cloak

Table-driven Routing Protocols

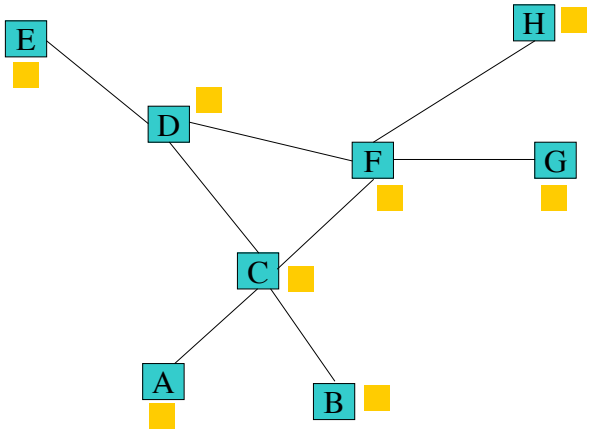
- These are "traditional" routing algorithms, trivially extended to the MANET scenario
  - Each node maintains one or more tables containing routing information to every other node in the network
  - All nodes updates these tables so as to maintain a consistent and up-to-date view of the network
- Several variations on the theme
  - LSR (Link State Routing protocol), OSR (Optimized link State Routing protocol), DSDV (Destination Sequenced Distance Vector), etc.
- The LSR example:
  - Each node keeps track of one-hop neighbors
  - This information is broadcasted through all the network
  - Each node has thus a global view of the network topology
  - Dijkstra algorithm is used to find route

The LSR Example

(Link State Routing Protocol)

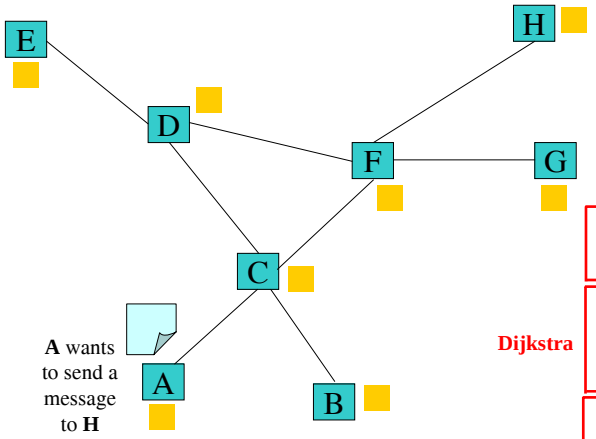


The LSR Example
(Link State Routing Protocol)



A: C
B: C
C: A, B, D, F
D: C, E, F
E: D
F: C, D, G, H
G: F
H: F

The LSR Example
(Link State Routing Protocol)

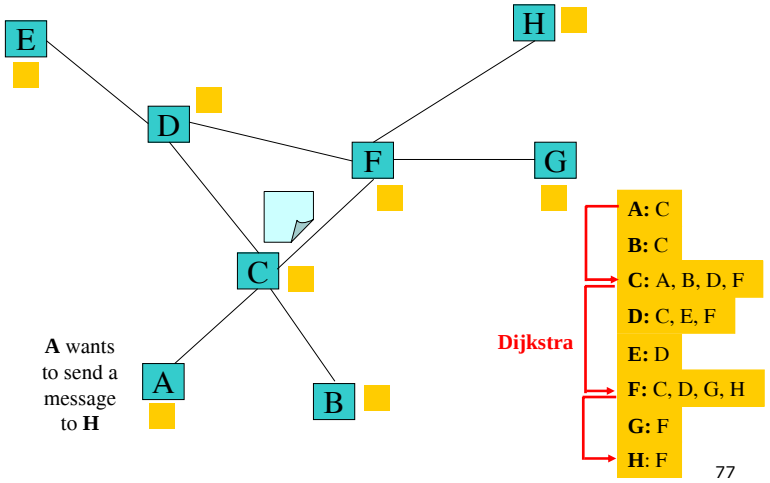


A wants to send a message to **H**

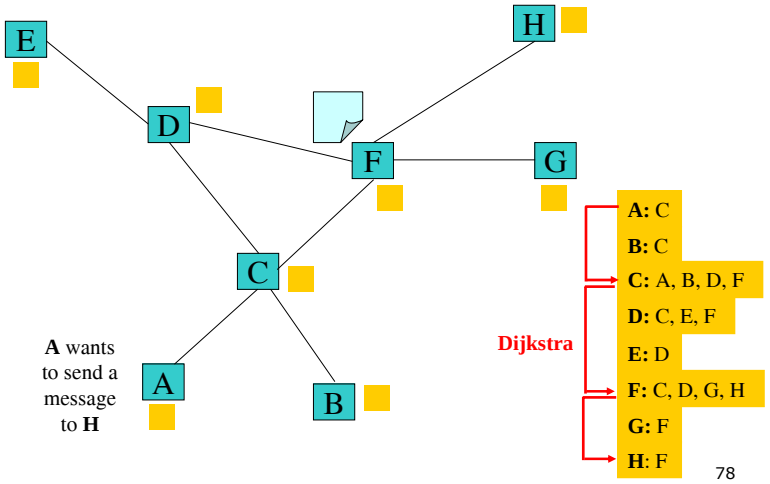
Dijkstra

A: C
B: C
C: A, B, D, F
D: C, E, F
E: D
F: C, D, G, H
G: F
H: F

The LSR Example
(Link State Routing Protocol)

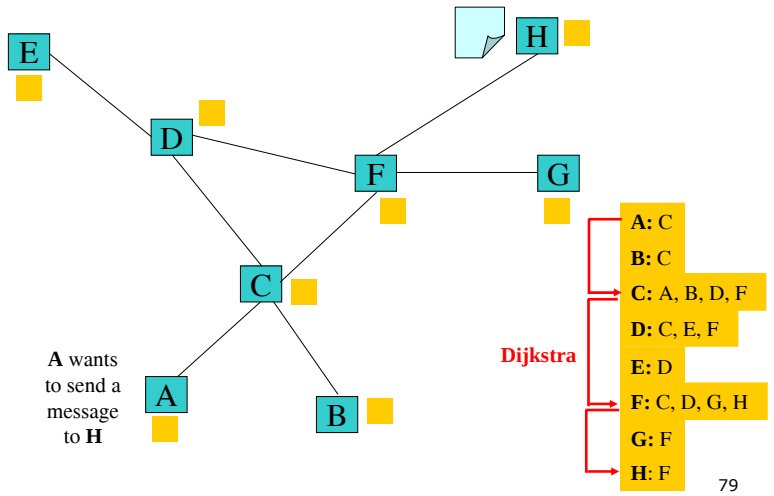


The LSR Example
(Link State Routing Protocol)



The LSR Example

(Link State Routing Protocol)



LSR: Pros & Cons

- Pros
  - Optimal Routing
  - Low latency to route packets
- Cons
  - Lot of data structures stored on each node (not suitable for small devices and large networks)
  - Lot of messages sent in the network
- And, **MORE IMPORTANT**
  - Suitable only in systems with very low dynamics (need to keep up with nodes' relocations)
  - Little to do with the needs of location-based routing (as in the cloak and in sensor networks)

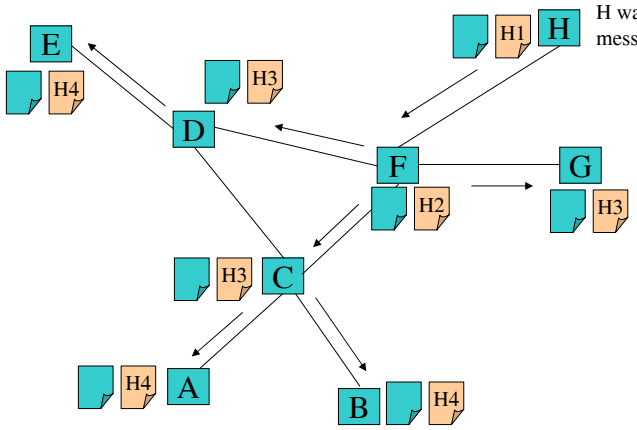
On Demand Routing Protocols

- No route information is maintained
  - routes are created on need, when a message has to be sent
  - When a source wants to send to a destination, it invokes a route discovery mechanism
- Several variations on the theme
  - DSR (Dynamic Source Routing protocol), CBRP (Cluster-Based Routing Protocol), Gradient Routing, etc.
- Example: **Gradient Routing**
  - Somehow inspired by ants and fields
  - Messages to create a sort of "pheromone" path to be used to temporarily store path from source to destination and backwards

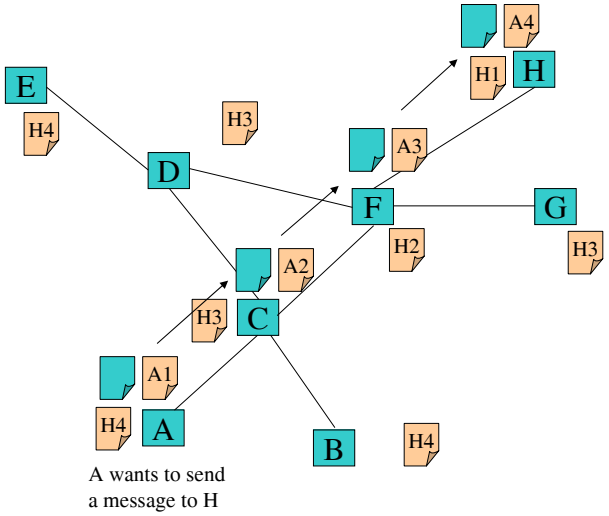
Gradient Routing

- The message used to create the overlay structure can be described as follows, in term of a **C**ontent and of a **P**ropagation (routing) rule:
  - *C=("structure", nodeName, hopCount)*
  - *P=(propagate to all nodes, increasing hopCount at every hop)*
- A message from a sender to a receiver, instead:
  - *C=("message", sender,receiver,message)*
  - *P=(if a structure tuple having my same receiver can be found follow downhill its hopCount, otherwise propagate to all nodes)*
- S*tructure* messages create a temporary overlay structure
  - a *message* tuple following downhill a *structure* tuple's *hopCount* can reach the node that created that particular structure, i.e, the receiver
  - When the structure information is absent, the routing simply reduces to flooding the network.

Gradient Routing: Example



Gradient Routing: Example



Gradient Routing: Pros & Cons

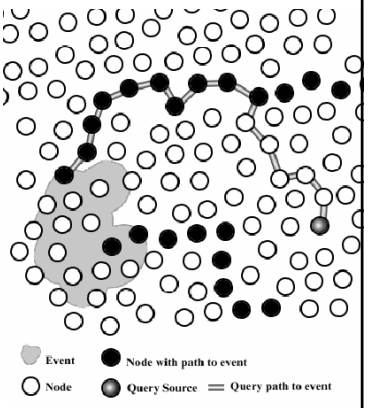
- Pros
  - Avoid constantly flooding the network
  - Reduced data structures stored on each node
- Cons
  - Routes are not ready when needed
  - High startup for messages
  - Not suited for highly dynamic networks
  - Requires adaptations to suit location-based routing

Adaptive Location-based Routing

- Location based routing + periodic update of coordinates to take into account devices' movements
- Requires:
  - Either fixed devices to act as central points for location reference. (e.g., in the cloak, a belt of a necklace)
  - Or some devices to be able to evaluate in an absolute way (e.g., GPS) their coordinates
  - So that other can compare
- Pros
  - Optimal routing and low latency
  - Never Flood
- Cons
  - Requires location information
  - Need external mechanism to gather location of partners
  - Not generally suited for distributed sensor networks

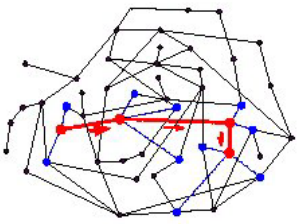
Rumor Routing and Gossip Algorithms

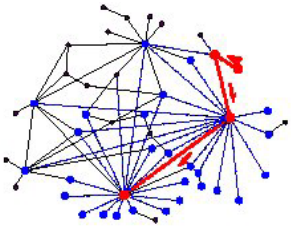
- Based on a simple yet effective idea
  - Let propagate in the network some sort of random "pheromone" path, leading backwards to a specific node or to a specific data
  - Let a sender (or some node in need of specific data) start randomly search in the network one of these paths
  - As soon as the random search meets a random path, it can then follow it backward
- Possible Applications:
  - Sensor networks (and the cloak of invisibility)
  - Publish-Subscribe and P2P computing on the Internet



Performances of Rumor Routing

- It's a probabilistic approach
  - The more the paths and the more extended is the search, the higher the probability to find a path soon
- Good performance in planar regula networks:
  - In a bounded region two lines have 69% probabilities to intersect…
  - So, if data is disseminated along 5 different paths, there is 99.7% of being found by a query path….
- In small world and scale free networks, the performances may be greatly enhanced
  - **Small-world** → it is easy to reach far nodes
  - **Clustering** → related nodes are greatly connected
  - **Power-Law** → queries and data are more likely to encounter





Pros & Cons of Rumor Routing

- Pros
  - Never Flood
  - Work incredibly well, against the apparently unreasonable starting assumptions
  - Everything is dynamic, so that mobility of nodes is tolerated without any pain
- Cons
  - Not certain performances
  - Strongly depends on the structural properties of networks

Back to the Invisible Cloak Hardware Issues

- Size of devices does not change, however:
  - **CONS** More bandwidth required to deal with coordinates update and more complicated routing
    - Calculations shows that some Terabyte of bandwidth per component may be required
    - Hard to say if this will be ever possible
  - **PROS** Cloak movement or human body thermal energy can be used to recharge cloak's devices
- How much for an Invisible Cloak ?!
  - The Institute for Defense Analysis has predicted that MEMS cost in the near future will reach **1 Euro** each.
  - A cloak of invisibility of $3m^2$ would require approximately *372,000* compound (multidirectional) devices to be invisible at a *10m* distance, implying an overall cost below a **half-million Euros/Dollars**.

The Invisible Cloak: Applications

- Besides obvious military applications…
  - Invisible soldiers?
  - Are we really sure these are not already at work? ;-(
- Painting the belly, drinking a "smart drug"
  - And then immediately visualize on the belly what's happening inside our stomach and our body
  - Also of great use in non-intrusive surgery, as a specific technology for augmented reality
- If we assume some small "storing" capability in device
  - We could use them to visualize a posteriori the structure of a hard to be accessed internal or pipeline
  - By simply injecting some "smart fluid" in the internal of the pipeline
- With reference to the "T-1000" vision
  - Other than capability of dynamic and adaptive self-assembly
  - There is need for dynamic image assembling

Conclusions and Open Issues

- The Cloak of Invisibility
  - We do not know if it will ever be realized in this terms
  - Still, we think it is interesting to shows various types of self-organization at work
- And also show that modern technologies plus self-organization
  - Can open a wide range of innovative applications
  - And can be a dramatic source of innovation
  - Even without reaching the extremes of the spray computers vision and of the invisibility cloak